Lab 8

Lab 7 cleaned up the architecture — singletons gone, frame loop in the right place, dependencies flowing in one direction. Lab 8 builds on that foundation by rethinking how animations and object lifetimes are modelled. The bespoke `AnimatedGameObject` is replaced by a reusable `SpriteSheet` that can hold multiple named animations, and a new `TemporaryGameObject` cleanly represents anything that should disappear after a fixed time. Input gets an event-driven facelift, and several classes lose the awkward "Logic" suffix.

Renames — clearer names, smaller surface

Three classes are renamed to drop unnecessary words:

- `GameCamera` → `Camera`
- `GameLogic` → `Engine`
- `InputLogic` → `Input`

These are pure renames — no behaviour changes — but they signal a shift in how the codebase is being framed. `Engine` is more accurate than `GameLogic` for a class that orchestrates the entire frame loop, and `Input` reads more naturally than `InputLogic` for something that just reads keyboard and mouse state. The `Game-` prefix only stays where it disambiguates against framework types (`GameRenderer`, `GameWindow`, `GameObject`).

SpriteSheet — animations as data

The new `Models/SpriteSheet.cs` is the centerpiece of this lab. It replaces `AnimatedGameObject` with a more flexible design: a sprite sheet object that holds a texture and a collection of named animations, any one of which can be activated at a time.

```
public class SpriteSheet
{
    public class Animation
    {
        public (int Row, int Col) StartFrame { get; init; }
        public (int Row, int Col) EndFrame { get; init; }
        public RendererFlip Flip { get; init; } = RendererFlip.None;
        public int DurationMs { get; init; }
        public bool Loop { get; init; }
    }

    public int RowCount { get; init; }
    public int ColumnCount { get; init; }

    public int FrameWidth { get; init; }
    public int FrameHeight { get; init; }
    public (int OffsetX, int OffsetY) FrameCenter { get; init; }

    public Animation? ActiveAnimation { get; private set; }
    public Dictionary<string, Animation> Animations { get; init; } = new();
```

```
    private int _textureId;
    private DateTimeOffset _animationStart = DateTimeOffset.MinValue;
```

The `Animation` nested class describes a single animation as data: where in the sheet it starts and ends, how long it should run, whether it loops, and whether it flips horizontally or vertically. A sprite sheet can contain any number of these, stored in a dictionary by name. This is a fundamentally different model from Lab 7's `AnimatedGameObject`, which baked the animation parameters into the constructor and could only play one fixed sequence.

The constructor loads the texture and records the sheet's geometry:

```
public SpriteSheet(GameRenderer renderer, string fileName, int rowCount, int
columnCount, int frameWidth,
    int frameHeight, (int OffsetX, int OffsetY) frameCenter)
{
    _textureId = renderer.LoadTexture(fileName, out var textureData);

    RowCount = rowCount;
    ColumnCount = columnCount;
    FrameWidth = frameWidth;
    FrameHeight = frameHeight;
    FrameCenter = frameCenter;
}
```

`FrameCenter` is the offset from the top-left of a frame to its logical anchor point — this lets the renderer position the sprite by where it conceptually "is" (typically the feet for a character, or the center for an explosion) rather than by the corner of the bounding rectangle.

Animations are activated by name:

```
public void ActivateAnimation(string name)
{
    if (!Animations.TryGetValue(name, out var animation)) return;

    ActiveAnimation = animation;
    _animationStart = DateTimeOffset.Now;
}
```

This sets the active animation and resets the timer. Calling it again with a different name (or even the same name) restarts playback.

The render method is where the sprite sheet earns its keep:

```
public void Render(GameRenderer renderer, (int X, int Y) dest, double angle =
0.0, Point rotationCenter = new())
```

```
{
    if (ActiveAnimation == null)
    {
        renderer.RenderTexture(_textureId, new Rectangle<int>(0, 0, FrameWidth,
FrameHeight),
            new Rectangle<int>(dest.X - FrameCenter.OffsetX, dest.Y -
FrameCenter.OffsetY, FrameWidth, FrameHeight),
            RendererFlip.None, angle, rotationCenter);
    }
    else
    {
        var totalFrames = (ActiveAnimation.EndFrame.Row -
ActiveAnimation.StartFrame.Row) * ColumnCount +
            ActiveAnimation.EndFrame.Col - ActiveAnimation.StartFrame.Col;
        var currentFrame = (int)((DateTimeOffset.Now -
_animationStart).TotalMilliseconds /
                                 (ActiveAnimation.DurationMs /
(double)totalFrames));
        if (currentFrame > totalFrames)
        {
            if (ActiveAnimation.Loop)
            {
                _animationStart = DateTimeOffset.Now;
                currentFrame = 0;
            }
            else
            {
                currentFrame = totalFrames;
            }
        }

        var currentRow = ActiveAnimation.StartFrame.Row + currentFrame /
ColumnCount;
        var currentCol = ActiveAnimation.StartFrame.Col + currentFrame %
ColumnCount;

        renderer.RenderTexture(_textureId,
            new Rectangle<int>(currentCol * FrameWidth, currentRow *
FrameHeight, FrameWidth, FrameHeight),
            new Rectangle<int>(dest.X - FrameCenter.OffsetX, dest.Y -
FrameCenter.OffsetY, FrameWidth, FrameHeight),
            ActiveAnimation.Flip, angle, rotationCenter);
    }
}
```

When no animation is active, it draws the first frame as a static sprite. When an animation is playing, it computes the current frame from elapsed wall-clock time, wraps around if the animation loops, and clamps to the last frame if it doesn't. The `currentFrame / ColumnCount` and `currentFrame % ColumnCount` arithmetic walks through the sheet in row-major order, starting from `StartFrame`.

A subtle but important difference from `AnimatedGameObject`: the sprite sheet does not remove itself when an animation finishes. Non-looping animations just freeze on their last frame, and the *owning object*

decides what to do — which is where `TemporaryGameObject` comes in.

TemporaryGameObject — lifetime as a property

The new `Models/TemporaryGameObject.cs` represents anything that should disappear from the world after a fixed duration. It is dead simple:

```
public class TemporaryGameObject : RenderableGameObject
{
    public double Ttl { get; init; }
    public bool IsExpired => (DateTimeOffset.Now - _spawnTime).TotalSeconds >=
Ttl;

    private DateTimeOffset _spawnTime;

    public TemporaryGameObject(SpriteSheet spriteSheet, double ttl, (int X, int
Y) position, double angle = 0.0, Point rotationCenter = new())
        : base(spriteSheet, position, angle, rotationCenter)
    {
        Ttl = ttl;
        _spawnTime = DateTimeOffset.Now;
    }
}
```

`Ttl` (time-to-live) is set at construction. `IsExpired` is computed on demand by comparing the current time to the recorded spawn time. There is no `Update` method — the engine just polls `IsExpired` and removes the object if it returns `true`.

This is conceptually cleaner than Lab 7's approach, where `Update` returned a `bool` indicating whether to keep the object alive. That mixed two responsibilities (advancing state, signalling lifetime). Now lifetime is a property of certain objects, animation timing is the responsibility of the sprite sheet, and game logic stays simple.

RenderableGameObject — composition over inheritance

`RenderableGameObject` is dramatically simplified. Instead of holding texture handles and source/destination rectangles directly, it now holds a `SpriteSheet` and delegates rendering to it:

```
public class RenderableGameObject : GameObject
{
    public SpriteSheet SpriteSheet { get; set; }
    public (int X, int Y) Position { get; set; }
    public double Angle { get; set; }
    public Point RotationCenter { get; set; }

    public RenderableGameObject(SpriteSheet spriteSheet, (int X, int Y)
position, double angle = 0.0,
        Point rotationCenter = new())
        : base()
```

```
    {
        SpriteSheet = spriteSheet;
        Position = position;
        Angle = angle;
        RotationCenter = rotationCenter;
    }

    public virtual void Render(GameRenderer renderer)
    {
        SpriteSheet.Render(renderer, Position, Angle, RotationCenter);
    }
}
```

The `Update` method is gone entirely — there is nothing for the base class to update because animation state lives in the sprite sheet and lifetime state lives in subclasses like `TemporaryGameObject`. The class now reads as exactly what it is: "a thing in the world with a position, an angle, and a sprite sheet to draw."

This is a shift from inheritance (`AnimatedGameObject extends RenderableGameObject`) to composition (`RenderableGameObject has-a SpriteSheet`). Adding new behaviours no longer requires a new subclass.

Input — events instead of polling

`Input` (formerly `InputLogic`) gets a major rework. It no longer holds a reference to the engine, and it no longer drives anything — instead, it exposes input state through query methods and raises an event for mouse clicks:

```
public unsafe class Input
{
    private readonly Sdl _sdl;

    public EventHandler<(int x, int y)>? OnMouseClick;

    public Input(Sdl sdl)
    {
        _sdl = sdl;
    }

    public bool IsLeftPressed()
    {
        ReadOnlySpan<byte> keyboardState = new(_sdl.GetKeyboardState(null),
(int)KeyCode.Count);
        return keyboardState[(int)KeyCode.Left] == 1;
    }

    public bool IsRightPressed() { /* same pattern */ }
    public bool IsUpPressed()    { /* same pattern */ }
    public bool IsDownPressed()  { /* same pattern */ }
```

This is a much cleaner separation of concerns. `Input` is now a passive query interface — it answers "is this key pressed right now?" — plus a single push notification for clicks. Whoever cares about input subscribes to the event or polls the methods.

The mouse handling inside `ProcessInput` becomes correspondingly simpler. Instead of recording coordinates and tracking button-down/button-up state across frames, it just fires the event when a primary click happens:

```
case (uint)EventType.Mousebuttondown:
{
    if (ev.Button.Button == (byte)MouseButton.Primary)
    {
        OnMouseClick?.Invoke(this, (ev.Button.X, ev.Button.Y));
    }

    break;
}
```

The button-up cases, the finger cases, and all the per-frame "if button is held, spawn a bomb" logic are gone. A single click produces a single event. This also fixes a subtle Lab 7 behaviour where holding the mouse button down would spawn a stream of bombs every frame.

Engine — wiring the new pieces together

`Engine` (formerly `GameLogic`) takes both the renderer and the input in its constructor, and subscribes to the click event during construction:

```
public Engine(GameRenderer renderer, Input input)
{
    _renderer = renderer;
    _input = input;

    _input.OnMouseClick += (_, coords) => AddBomb(coords.x, coords.y);
}
```

The lambda turns mouse clicks directly into bomb spawns. Because `AddBomb` is now triggered by an event rather than checked every frame, it has been made `private` — no other class needs to call it.

The `InitializeGame` method is renamed to `SetupWorld`, which describes its purpose better. The body is unchanged except for the player constructor (which no longer needs an ID).

`ProcessFrame`, which had been empty since Lab 4, now has a real job. It samples input and updates the player:

```
public void ProcessFrame()
{
    var currentTime = DateTimeOffset.Now;
```

```
    var msSinceLastFrame = (currentTime - _lastUpdate).TotalMilliseconds;
    _lastUpdate = currentTime;

    double up = _input.IsUpPressed() ? 1.0 : 0.0;
    double down = _input.IsDownPressed() ? 1.0 : 0.0;
    double left = _input.IsLeftPressed() ? 1.0 : 0.0;
    double right = _input.IsRightPressed() ? 1.0 : 0.0;

    _player?.UpdatePosition(up, down, left, right, (int)msSinceLastFrame);
}
```

The frame timing logic moves here too, since this is where player movement happens. `RenderFrame` no longer needs to track elapsed time — animations are now timed by their sprite sheets using wall-clock time, not by deltas passed down from the loop.

`RenderAllObjects` is correspondingly simpler. There is no more `Update` call, and lifetime checks are done by pattern-matching on `TemporaryGameObject`:

```
public void RenderAllObjects()
{
    var toRemove = new List<int>();
    foreach (var gameObject in GetRenderables())
    {
        gameObject.Render(_renderer);
        if (gameObject is TemporaryGameObject { IsExpired: true }
tempGameObject)
        {
            toRemove.Add(tempGameObject.Id);
        }
    }

    foreach (var id in toRemove)
    {
        _gameObjects.Remove(id);
    }

    _player?.Render(_renderer);
}
```

The C# pattern `gameObject is TemporaryGameObject { IsExpired: true }` does two checks at once: is this a temporary object, and is its `IsExpired` property `true`? Permanent objects (like things that aren't `TemporaryGameObject`) are simply ignored by this check and stay in the collection forever. This is much more general than Lab 7's "if Update returns false, remove it" — the engine no longer needs to know anything about animation duration or lifetime calculations.

The new `AddBomb` shows the new API in action:

```
private void AddBomb(int screenX, int screenY)
```

```
{
    var worldCoords = _renderer.ToWorldCoordinates(screenX, screenY);

    SpriteSheet spriteSheet = new(_renderer, Path.Combine("Assets",
"BombExploding.png"), 1, 13, 32, 64, (16, 48));
    spriteSheet.Animations["Explode"] = new SpriteSheet.Animation
    {
        StartFrame = (0, 0),
        EndFrame = (0, 12),
        DurationMs = 2000,
        Loop = false
    };
    spriteSheet.ActivateAnimation("Explode");

    TemporaryGameObject bomb = new(spriteSheet, 2.1, (worldCoords.X,
worldCoords.Y));
    _gameObjects.Add(bomb.Id, bomb);
}
```

The bomb is now expressed declaratively. A sprite sheet is built from the explosion image (1 row × 13 columns of 32×64 frames, with the anchor point at offset (16, 48) — center horizontally, near the bottom vertically). An "Explode" animation is registered describing which frames to walk through, how long it takes, and whether to loop. The animation is activated, and a `TemporaryGameObject` wraps it with a 2.1-second TTL — slightly longer than the 2-second animation, to make sure the last frame is visible before the object disappears.

Compare this to Lab 7's bomb code, which crammed every parameter into the `AnimatedGameObject` constructor in a fixed order. The new form is more verbose but much more expressive — adding a second animation (a "Disarm" sequence, perhaps) would just mean adding another entry to the dictionary.

The `_bombIds` counter from earlier labs is finally gone — it was already vestigial after Lab 7's automatic ID assignment, and there is no remaining caller for it.

GameWindow — proper disposal

`GameWindow` now implements `IDisposable` with the standard disposal pattern, including a finalizer for safety:

```
public unsafe class GameWindow : IDisposable
{
    // ...

    private void ReleaseUnmanagedResources()
    {
        if (_window != IntPtr.Zero)
        {
            _sdl.DestroyWindow((Window*)_window);
            _window = IntPtr.Zero;
        }
    }
```

```
    public void Dispose()
    {
        ReleaseUnmanagedResources();
        GC.SuppressFinalize(this);
    }

    ~GameWindow()
    {
        ReleaseUnmanagedResources();
    }
}
```

The `_window` field is no longer `readonly` because `Dispose` needs to null it out to prevent double-free. The previous `Destroy()` method is replaced by `Dispose()`, which is the .NET-idiomatic name. The finalizer (`~GameWindow`) is a fallback that runs if someone forgets to call `Dispose` — `ReleaseUnmanagedResources` is idempotent, so calling it from both paths is safe.

Program.cs — using statement and new construction order

The main loop is updated to use the disposable window and the new construction order:

```
using (var gameWindow = new GameWindow(sdl))
{
    var input = new Input(sdl);
    var gameRenderer = new GameRenderer(sdl, gameWindow);
    var engine = new Engine(gameRenderer, input);

    engine.SetupWorld();

    bool quit = false;
    while (!quit)
    {
        quit = input.ProcessInput();
        if (quit) break;

        engine.ProcessFrame();
        engine.RenderFrame();

        Thread.Sleep(13);
    }
}

sdl.Quit();
```

The `using` statement guarantees the window is destroyed even if an exception escapes the loop. `Input` is constructed before the engine because the engine needs to subscribe to its events. The explicit `gameWindow.Destroy()` call from earlier labs is gone, replaced by the `using` block's automatic disposal.

Summary

By the end of Lab 8, the codebase has crossed a meaningful line. Animations are no longer hard-coded into specific object types — they live in reusable sprite sheets that can hold many named sequences. Object lifetimes are no longer entangled with animation timing — `TemporaryGameObject` expresses "this disappears after N seconds" as its own concern. Input is no longer polled-and-acted-upon in a single function — it exposes state and raises events that other systems subscribe to. Resource disposal follows .NET conventions. The naming is cleaner, the responsibilities are sharper, and the bomb spawn site reads like a description of what should happen rather than a tightly-packed parameter list. The game itself looks the same, but the code that produces it is substantially more capable.